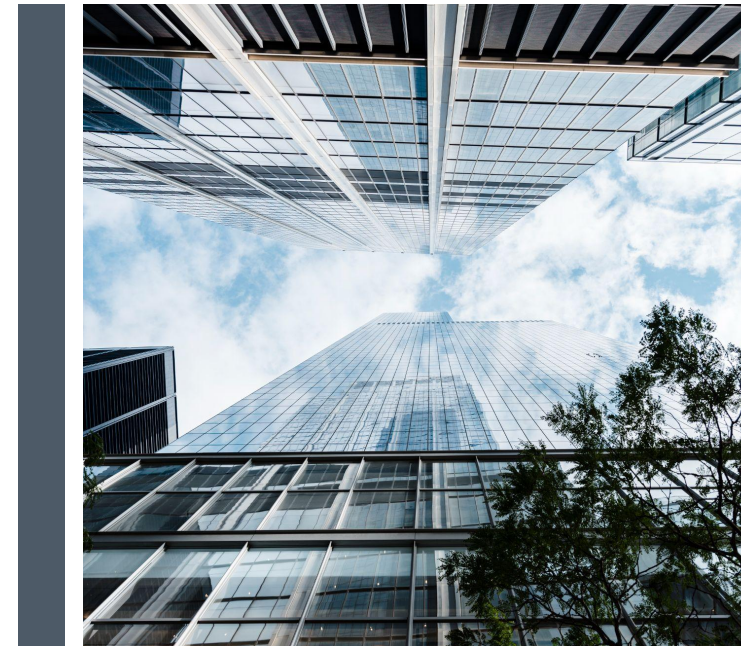
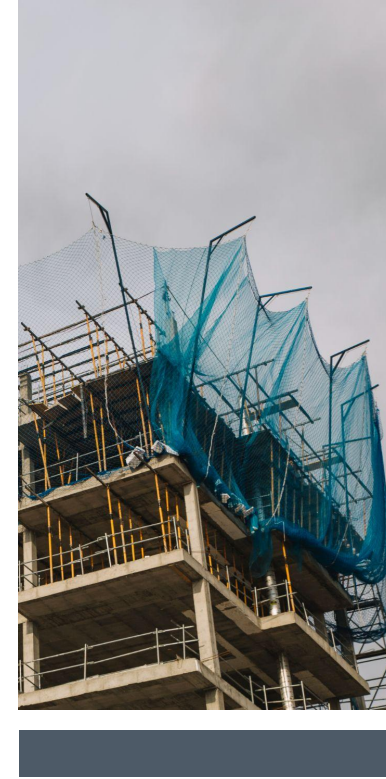
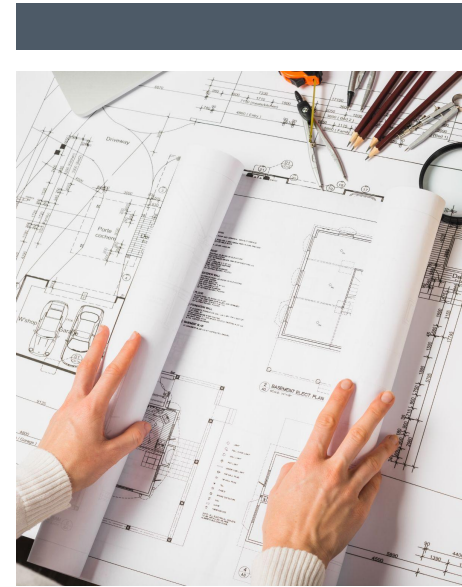


Software Engineering Quality Assurance

Software Engineering Course – 7
2025-2026





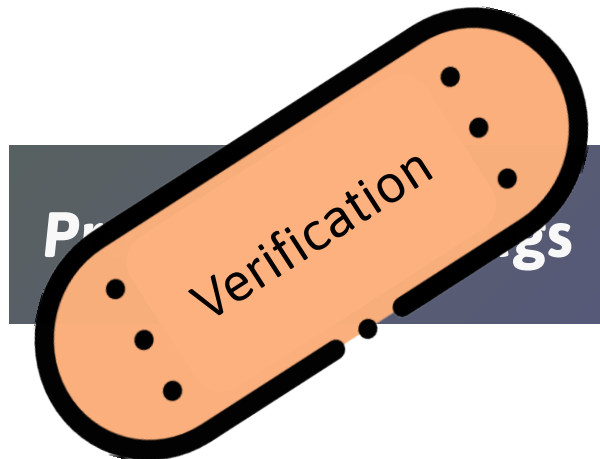
Introduction to Software Testing

- Verification & Validation
- Software Testing Types
- Software Testing Processes
- Unit Testing
- Performance Testing
- Record & Playback
- Designing a Testing Process
- Software Engineer & Testing



What is Software Testing?

- Software Testing is a **process** in which you **execute** your software with **simulated** data.
- If the test passes, the program is correct for the given data.
 - If the data is representative for a larger class of data, you can infer that the program is correct for any member of the data set.
 - If the test fails, we have **a bug**
- Two types of bugs:





Verification & Validation

“Testing can only show the presence of errors, not their absence[†]”

- Verification – Testing that the program runs correctly
 - $2 + 2 = 4$ – True
- Validation – Testing that the program does what it is supposed to do
 - You require a list of the top 10 IMDb movies, but you get a queue of the top 10 IMDb movies

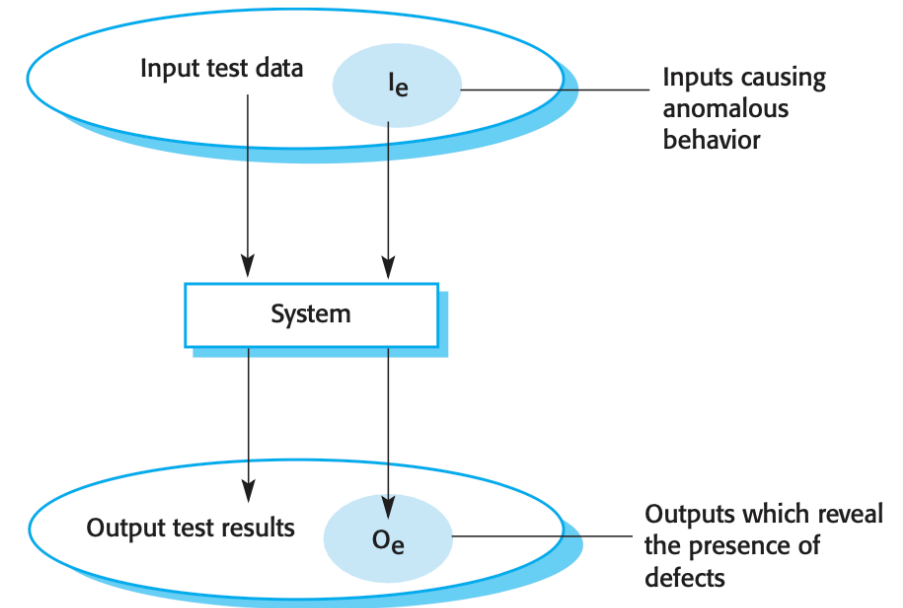
[†]Dijkstra, E. W. 1972. “The Humble Programmer.” Comm. ACM 15 (10): 859–66. doi:10.1145/355604.361591



Verification & Validation

“Testing can only show the presence of errors, not their absence[†]”

- Verification – Testing that the program runs correctly
 - $2 + 2 = 4$ – True
- Validation – Testing that the program does what it is supposed to do
 - You require a list of the top 10 IMDb movies, but you get a queue of the top 10 IMDb movies



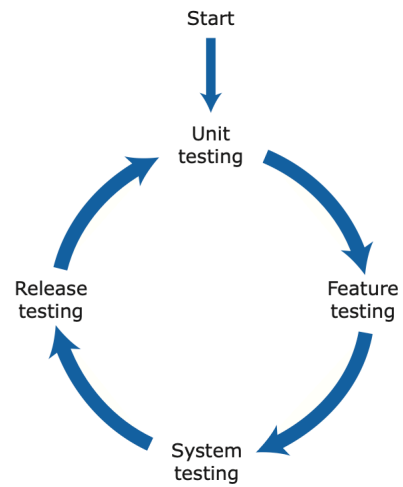
■ **Validation:** *Are we building the right product?*

■ **Verification:** *Are we building the product right?*

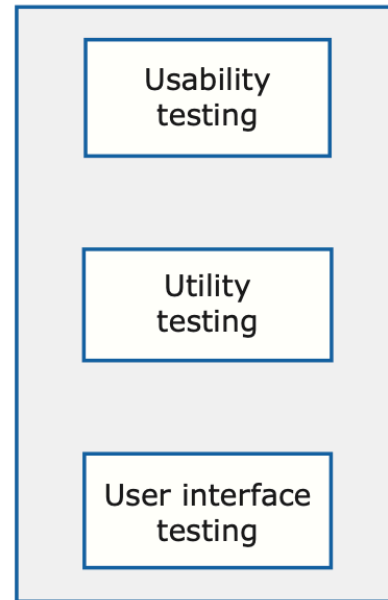
[†]Dijkstra, E. W. 1972. “The Humble Programmer.” Comm. ACM 15 (10): 859–66. doi:10.1145/355604.361591

Testing Types

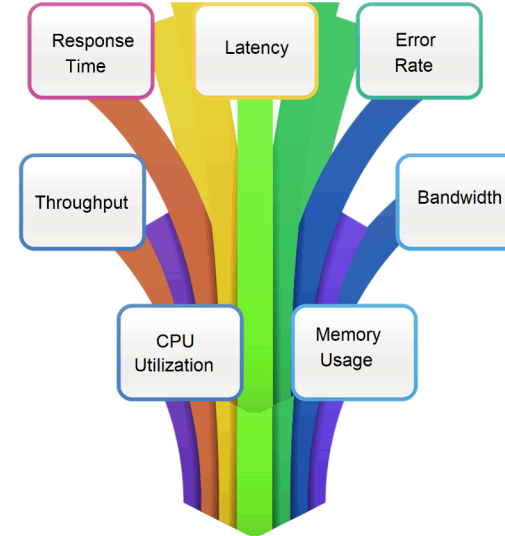
Functional testing



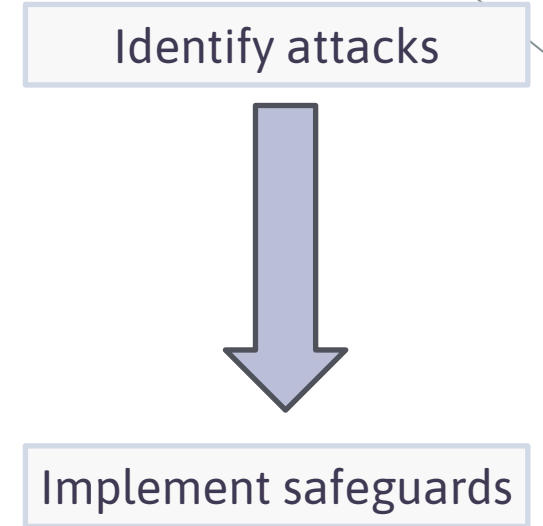
User testing



Performance testing

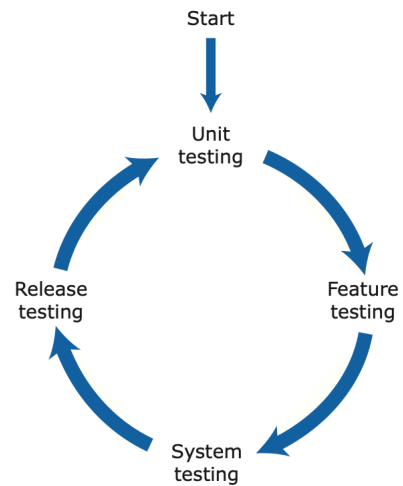


Security testing



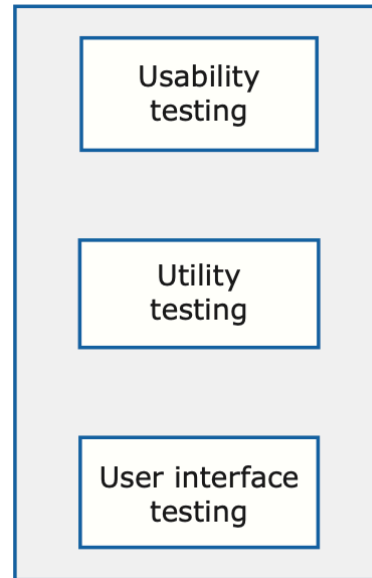
Testing Types

Functional testing



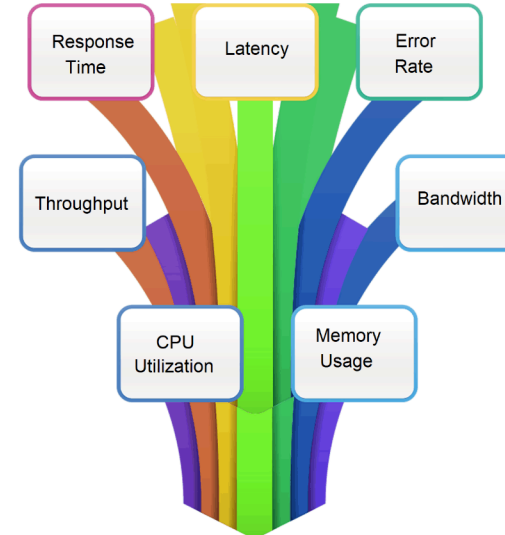
V & V

User testing



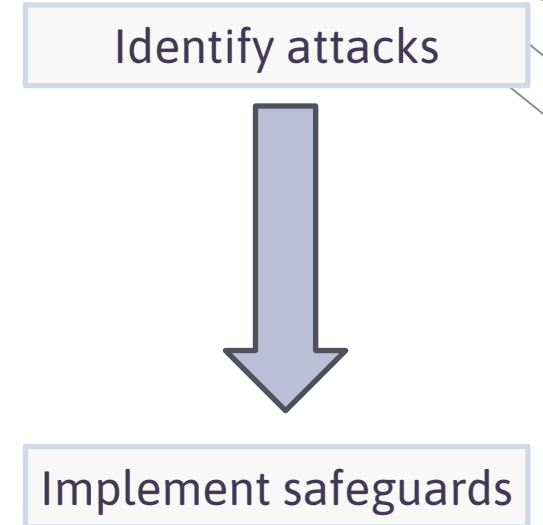
Validation

Performance testing



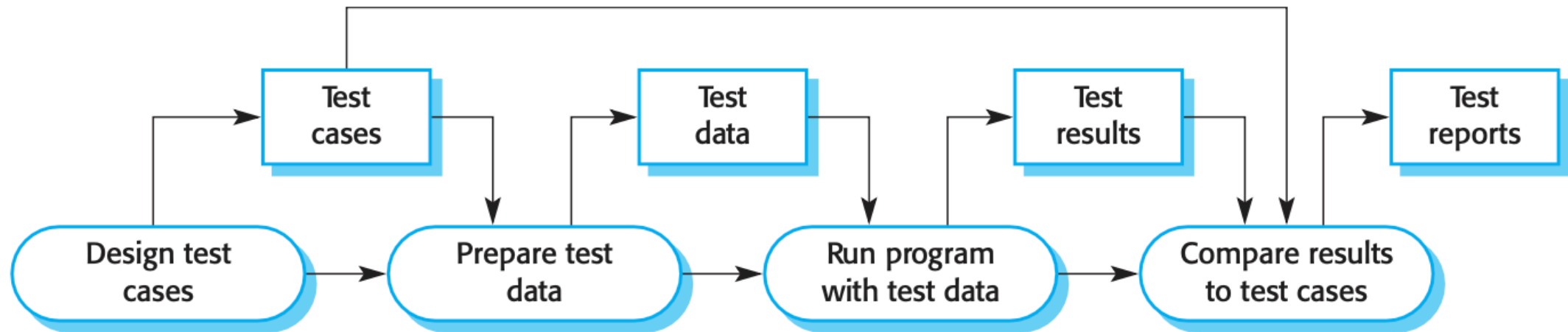
Verification

Security testing

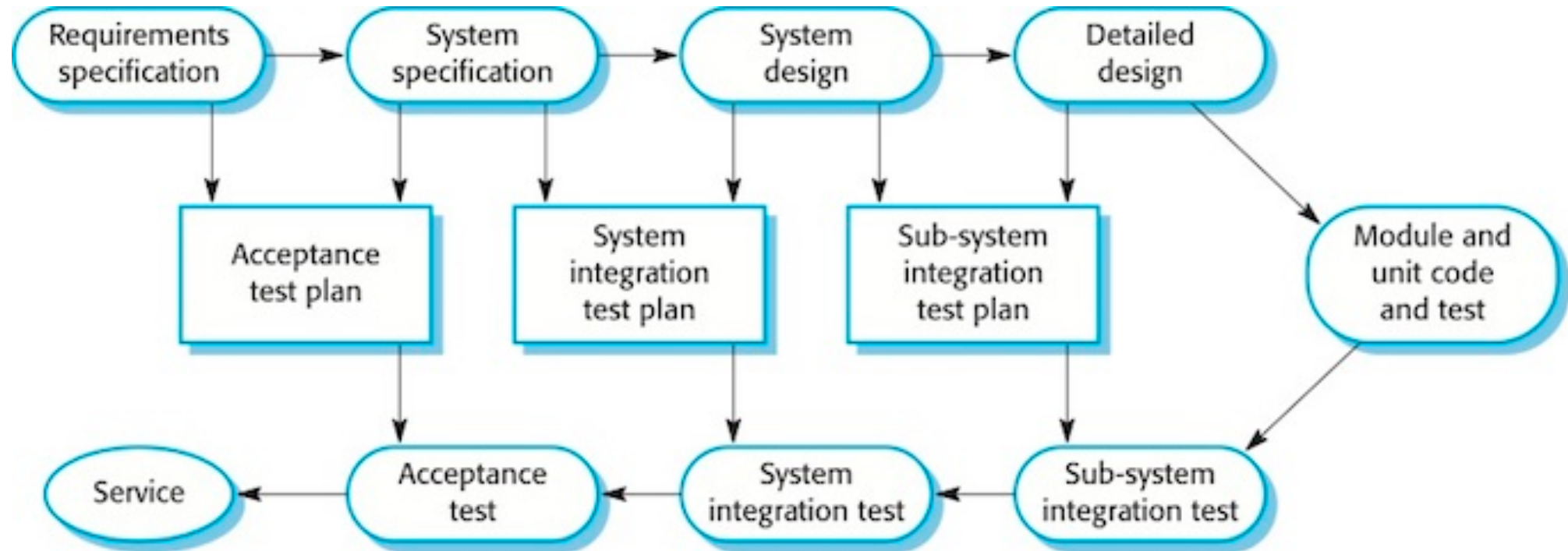


V & V

Basic Software Testing Process



V-model Testing Process





Test Driven Development





Unit Testing

- Defining and testing the smallest unit of your software.
 - A unit is small enough to be a unit when you won't reduce the complexity of the test any further.
- Stop making the Unit too small.
- When to use it?
 - When you are able to define a unit (equivalent sizes, not necessarily indivisible)
 - When the unit is complex or critical.
- When not to use it?
 - When the "unit" depends on many other services (for example if you have to mock multiple dependencies – DB connection, DTO TranslationService, etc.)
 - When test assertion depends on too many variables (but then again, probably your unit could be broken down) – see next slides



Unit Testing (Bad Example)

Depends on a complex(?)
Service function

[HttpGet]

👤 Rares Cristea

```
public async Task<ActionResult<List<Teacher>>> GetAllTeachers()
{
    var result:List<TeacherSendDTO>? = await _teacherService.GetAllTeachers();
    if (result is null)
    {
        return ⚙️ NotFound("Didn't find the list of teachers.");
    }
    return ⚙️ Ok(result);
}
```

Not a lot of actual
functionality

Heavily dependent on
Database Scenario

Unit Testing (Better Example)

It's good to make a Unit Test for this function, because:

- Single, well defined purpose
- Can easily define homogenous test cases (departments, and corresponding DTOs)

```
public class Department : BaseModel
{
    [5 usages]
    public long Id { get; set; }

    [4 usages]
    public string FullName { get; set; }

    [5 usages]
    public long FacultyId { get; set; }

    [1 usage]
    public Faculty Faculty { get; set; }

    [2 usages]
    public ICollection<TeacherDepartmentMembership> TeacherDepartmentMemberships { get; set; }

    [1 usage]
    public ICollection<Teacher> Teachers { get; set; }
}
```

```
public class DepartmentSendDTO
{
    [2 usages]
    public long Id { get; set; }

    [2 usages]
    public string FullName { get; set; }

    [2 usages]
    public FacultySendDTO Faculty { get; set; }
}
```

```
public DepartmentSendDTO DepartmentToDepartmentSendDto(Department department)
{
    return new DepartmentSendDTO()
    {
        Id = department.Id,
        FullName = department.FullName,
        Faculty = _facultyService.GetSingleFaculty(department.FacultyId).Result
    };
}
```

Still not ideal due to
this dependency

Unit test breakdown

```
class Program
{
    static void Main(string[] args)
    {
        using (SqlConnection connection = new SqlConnection(
            "Server=[SQ_SIXTEEN];Database=[PocketCentral];User ID=[un];Password=[pw];Trusted_Connection=true"))
        {
            connection.Open();
            using (SqlCommand command = new SqlCommand("SELECT * FROM tbl_Terminal", connection))
            {
                using (SqlDataReader reader = command.ExecuteReader())
                {
                    while (reader.Read())
                    {
                        for (int i = 0; i < reader.FieldCount; i++)
                        {
                            Console.WriteLine(reader.GetValue(i));
                        }
                        Console.WriteLine();
                    }
                }
            }
        }
    }
}
```

- Let's assume the code above.
- It opens a **DB connection**, **reads** some data, and **displays** it in the console.



Unit test breakdown

What can go wrong?

Database may not connect

Parsing a non-iterable object

SQL malformed

Trying to write an object without writing overload

```
class Program
{
    static void Main(string[] args)
    {
        using (SqlConnection connection = new SqlConnection(
            "Server=[SQ_SIXTEEN];Database=[PocketCentral];User ID=[un];Password=[pw];Trusted_Connection=true"))
        {
            connection.Open();
            using (SqlCommand command = new SqlCommand("SELECT * FROM tbl_Terminal", connection))
            {
                Are these versions of the same problem?
                for (int i = 0; i < reader.FieldCount; i++)
                {
                    Console.WriteLine(reader.GetValue(i));
                }
                Console.WriteLine();
            }
        }
    }
}
```

Unit test breakdown

+ Connecting to the database

+ Executing an SQL command

+ Reading, and parsing an object

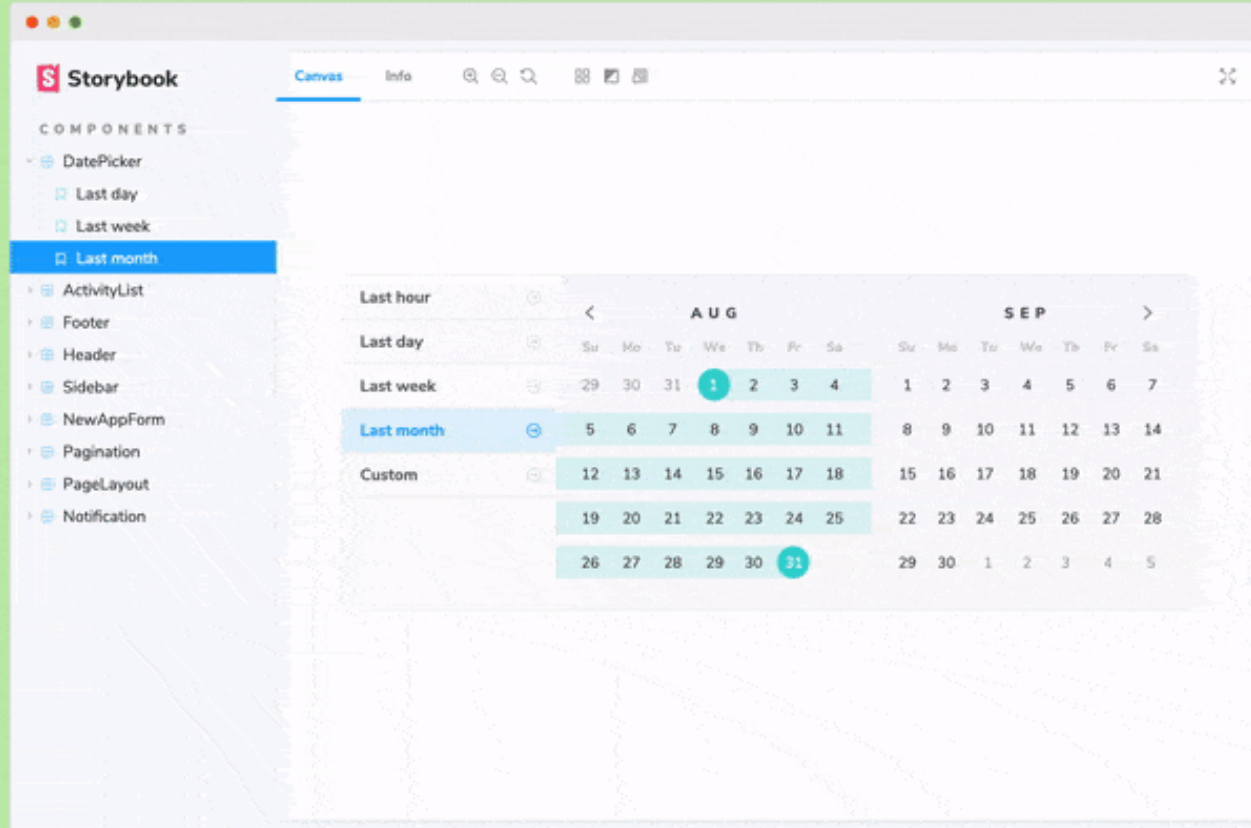
+ Writing

4 Unit Tests

(each with multiple test values)

```
class Program
{
    static void Main(string[] args)
    {
        using (SqlConnection connection = new SqlConnection(
            "Server=[SQ_SIXTEEN];Database=[PocketCentral];User ID=[un];Password=[pw];Trusted_Connection=true"))
        {
            connection.Open();
            using (SqlCommand command = new SqlCommand("SELECT * FROM tbl_Terminal", connection))
            {
                using (SqlDataReader reader = command.ExecuteReader())
                {
                    while (reader.Read())
                    {
                        for (int i = 0; i < reader.FieldCount; i++)
                        {
                            Console.WriteLine(reader.GetValue(i));
                        }
                        Console.WriteLine();
                    }
                }
            }
        }
    }
}
```

Visual "Unit Testing"



Visual test in a glance





Performance Testing

- Load test
- Stress Test
- Scalability Test
- Endurance Testing
- Great explanation: (1) Performance, Load, Stress or Endurance Test? Which do you want? | LinkedIn



Performance Testing

The screenshot shows the Postman interface with a workspace named 'vocea-universitati'. The 'Runs' tab is selected, displaying a performance test run for the collection 'vocea-universitati'. The test was executed on December 18, 2023, at 13:08:37, with 20 virtual users (VUs) for a duration of 1 minute. The results show 251 total requests, a throughput of 10.64 requests per second, an average response time of 1,159 ms, and an error rate of 84.06%.

Run	Start time	VUs	Duration	Total requests	Requests/s	Resp. time (Avg ms)	Error %
#1	Dec 18, 2023, 13:08:37	20	1m	251	10.64	1,159	84.06

The screenshot shows a code editor displaying a stack trace for a runtime error. The error is a `System.InvalidOperationException` with the message: "An exception has been raised that is likely due to a transient failure." The stack trace indicates the error occurred in the `PostgresSQL.Storage.Internal.NpgsqlExecutionStrategy` class, specifically in the `ExecuteAsync` method. The error is related to a PostgreSQL connection timeout.

```
at Microsoft.EntityFrameworkCore.Query.Internal.SingleQueryingEnumerable`1.AsyncEnumerator.MoveNextAsync()
at Microsoft.EntityFrameworkCore.EntityFrameworkQueryableExtensions.ToListAsync[TSource](IQueryable`1 source, CancellationToken cancellationToken)
at Microsoft.EntityFrameworkCore.EntityFrameworkQueryableExtensions.ToListAsync[TSource](IQueryable`1 source, CancellationToken cancellationToken)
at VoceaUniversitatii.Services.FacultyService.FacultyService.GetAllFaculties() in C:\Users\Rares\RiderProjects\vocea-universitatii\vocea-universitatii\Services\FacultyService\FacultyService.cs:line 19
at VoceaUniversitatii.Controllers.FacultiesController.GetAllFaculties() in C:\Users\Rares\RiderProjects\vocea-universitatii\vocea-universitatii\Controllers\FacultiesController.cs:line 22
at Microsoft.AspNetCore.Mvc.Infrastructure.ActionMethodExecutor.AwaitableObjectResultExecutor.Execute(ActionContext actionContext, IActionResultTypeMapper mapper, ObjectMethodExecutor executor, Object controller, Object[] arguments)
at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.<InvokeActionMethodAsync>g__Awaited|12_0(ControllerActionInvoker invoker, ValueTask`1 actionResultValueTask)
at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.<InvokeNextActionFilterAsync>g__Awaited|10_0(ControllerActionInvoker invoker, Task lastTask, State next, Scope scope, Object state, Boolean isCompleted)
at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.Rethrow(ActionExecutedContextSealed context)
at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.Next(State& next, Scope& scope, Object& state, Boolean& isCompleted)
at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.<InvokeInnerFilterAsync>g__Awaited|13_0(ControllerActionInvoker invoker, Task lastTask, State next, Scope scope, Object state, Boolean isCompleted)
at Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.<InvokeFilterPipelineAsync>g__Awaited|20_0(ResourceInvoker invoker, Task lastTask, State next, Scope scope, Object state, Boolean isCompleted)
at Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.<InvokeAsync>g__Awaited|17_0(ResourceInvoker invoker, Task task, IDisposable scope)
at Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.<InvokeAsync>g__Awaited|17_0(ResourceInvoker invoker, Task task, IDisposable scope)
at Microsoft.AspNetCore.Routing.EndpointMiddleware.<Invoke>g__AwaitRequestTask|6_0(Endpoint endpoint, Task requestTask, ILogger logger)
at Microsoft.AspNetCore.Authorization.AuthorizationMiddleware.Invoke(HttpContext context)
at Swashbuckle.AspNetCore.SwaggerUI.SwaggerUIMiddleware.Invoke(HttpContext httpContext)
at Swashbuckle.AspNetCore.Swagger.SwaggerMiddleware.Invoke(HttpContext httpContext, ISwaggerProvider swaggerProvider)
at Microsoft.AspNetCore.Authentication.AuthenticationMiddleware.Invoke(HttpContext context)
at Microsoft.AspNetCore.Diagnostics.DeveloperExceptionPageMiddlewareImpl.Invoke(HttpContext context)
Fail: Microsoft.AspNetCore.Diagnostics.DeveloperExceptionPageMiddleware[1]
An unhandled exception has occurred while executing the request.
System.InvalidOperationException: An exception has been raised that is likely due to a transient failure.
----> Npgsql.NpgsqlException (0x80004005): Failed to connect to 13.93.108.65:5432
----> System.Net.Sockets.SocketException (10061): No connection could be made because the target machine actively refused it.
at System.Net.Sockets.Socket.AwaitableSocketAsyncEventArgs.ThrowException(SocketError error, CancellationToken cancellationToken)
at System.Net.Sockets.Socket.AwaitableSocketAsyncEventArgs.System.Threading.Tasks.Sources.IValueTaskSource.GetResult(Int16 token)
at System.Net.Sockets.Socket.<ConnectAsync>g__WaitForConnectWithCancellation|281_0(AwaitableSocketAsyncEventArgs saea, ValueTask connectTask, CancellationToken cancellationToken)
at Npgsql.TaskTimeoutAndCancellation.ExecuteAsync(Func`2 getTaskFunc, NpgsqlTimeout timeout, CancellationToken cancellationToken)
at Npgsql.Internal.NpgsqlConnector.ConnectAsync(NpgsqlTimeout timeout, CancellationToken cancellationToken)
at Npgsql.Internal.NpgsqlConnector.RawOpen(SslMode sslMode, NpgsqlTimeout timeout, Boolean async, CancellationToken cancellationToken, Boolean isFirstAttempt)
at Npgsql.Internal.NpgsqlConnector.<Open>g__OpenCore|216_1(NpgsqlConnector conn, SslMode sslMode, NpgsqlTimeout timeout, Boolean async, CancellationToken cancellationToken, Boolean isFirstAttempt)
at Npgsql.Internal.NpgsqlConnector.Open(NpgsqlTimeout timeout, Boolean async, CancellationToken cancellationToken)
at Npgsql.PoolingDataSource.OpenNewConnector(NpgsqlConnection conn, NpgsqlTimeout timeout, Boolean async, CancellationToken cancellationToken)
at Npgsql.PoolingDataSource.<Get>g__RentAsync|28_0(NpgsqlConnection conn, NpgsqlTimeout timeout, Boolean async, CancellationToken cancellationToken)
at Npgsql.NpgsqlConnection.<Open>g__OpenAsync|145_0(Boolean async, CancellationToken cancellationToken)
at Microsoft.EntityFrameworkCore.Storage.RelationalConnection.OpenInternalAsync(Boolean errorsExpected, CancellationToken cancellationToken)
at Microsoft.EntityFrameworkCore.Storage.RelationalConnection.OpenInternalAsync(Boolean errorsExpected, CancellationToken cancellationToken)
at Microsoft.EntityFrameworkCore.Storage.RelationalCommand.ExecuteReaderAsync(RelationalCommandParameterObject parameterObject, CancellationToken cancellationToken)
at Microsoft.EntityFrameworkCore.Query.Internal.SingleQueryingEnumerable`1.AsyncEnumerator.InitializeReaderAsync(AsyncEnumerator enumerator, CancellationToken cancellationToken)
at Npgsql.EntityFrameworkCore.PostgreSQL.Storage.Internal.NpgsqlExecutionStrategy.ExecuteAsync[TState,TResult](TState state, Func`4 operation, Func`4 verifySucceeded, CancellationToken cancellationToken)
--- End of inner exception stack trace ---
at Npgsql.EntityFrameworkCore.PostgreSQL.Storage.Internal.NpgsqlExecutionStrategy.ExecuteAsync[TState,TResult](TState state, Func`4 operation, Func`4 verifySucceeded, CancellationToken cancellationToken)
```

Performance Testing

The screenshot shows the Postman application interface. On the left, the 'My Workspace' sidebar displays a collection of API endpoints for 'vocea-universitati', including endpoints for Departments, Disciplines, Faculties, and Teachers. The main area is divided into two panes. The left pane, titled 'Run order', lists the endpoints to be tested, with checkboxes for selecting them. The right pane, titled 'Performance', shows the 'Test how your APIs perform under load' dialog. This dialog includes a 'Virtual users' field set to 2, a 'Test duration' field set to 1 minute, and a 'Load profile' dropdown set to 'Fixed'. A 'Run' button is visible at the bottom of the performance setup area.

The screenshot shows a code editor window displaying a series of SQL queries and database commands. The queries are organized into sections, each starting with an 'info:' header. The first section shows a complex JOIN query involving 'Teachers', 'Departments', and 'TeacherDepartmentMembership' tables. The second section shows a query for 'Faculties' with a 'LIMIT 1' clause. The third section shows a query for 'Teachers' with a 'LIMIT 1' clause. The fourth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The fifth section shows a query for 'Teachers' with a 'LIMIT 1' clause. The sixth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The seventh section shows a query for 'Teachers' with a 'LIMIT 1' clause. The eighth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The ninth section shows a query for 'Teachers' with a 'LIMIT 1' clause. The tenth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The eleventh section shows a query for 'Teachers' with a 'LIMIT 1' clause. The twelfth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The thirteenth section shows a query for 'Teachers' with a 'LIMIT 1' clause. The fourteenth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The fifteenth section shows a query for 'Teachers' with a 'LIMIT 1' clause. The sixteenth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The seventeenth section shows a query for 'Teachers' with a 'LIMIT 1' clause. The eighteenth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The nineteenth section shows a query for 'Teachers' with a 'LIMIT 1' clause. The twentieth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The twenty-first section shows a query for 'Teachers' with a 'LIMIT 1' clause. The twenty-second section shows a query for 'Faculties' with a 'LIMIT 1' clause. The twenty-third section shows a query for 'Teachers' with a 'LIMIT 1' clause. The twenty-fourth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The twenty-fifth section shows a query for 'Teachers' with a 'LIMIT 1' clause. The twenty-sixth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The twenty-seventh section shows a query for 'Teachers' with a 'LIMIT 1' clause. The twenty-eighth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The twenty-ninth section shows a query for 'Teachers' with a 'LIMIT 1' clause. The thirtieth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The thirty-first section shows a query for 'Teachers' with a 'LIMIT 1' clause. The thirty-second section shows a query for 'Faculties' with a 'LIMIT 1' clause. The thirty-third section shows a query for 'Teachers' with a 'LIMIT 1' clause. The thirty-fourth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The thirty-fifth section shows a query for 'Teachers' with a 'LIMIT 1' clause. The thirty-sixth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The thirty-seventh section shows a query for 'Teachers' with a 'LIMIT 1' clause. The thirty-eighth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The thirty-ninth section shows a query for 'Teachers' with a 'LIMIT 1' clause. The fortieth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The forty-first section shows a query for 'Teachers' with a 'LIMIT 1' clause. The forty-second section shows a query for 'Faculties' with a 'LIMIT 1' clause. The forty-third section shows a query for 'Teachers' with a 'LIMIT 1' clause. The forty-fourth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The forty-fifth section shows a query for 'Teachers' with a 'LIMIT 1' clause. The forty-sixth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The forty-seventh section shows a query for 'Teachers' with a 'LIMIT 1' clause. The forty-eighth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The forty-ninth section shows a query for 'Teachers' with a 'LIMIT 1' clause. The fiftieth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The fifty-first section shows a query for 'Teachers' with a 'LIMIT 1' clause. The fifty-second section shows a query for 'Faculties' with a 'LIMIT 1' clause. The fifty-third section shows a query for 'Teachers' with a 'LIMIT 1' clause. The fifty-fourth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The fifty-fifth section shows a query for 'Teachers' with a 'LIMIT 1' clause. The fifty-sixth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The fifty-seventh section shows a query for 'Teachers' with a 'LIMIT 1' clause. The fifty-eighth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The fifty-ninth section shows a query for 'Teachers' with a 'LIMIT 1' clause. The sixtieth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The sixty-first section shows a query for 'Teachers' with a 'LIMIT 1' clause. The sixty-second section shows a query for 'Faculties' with a 'LIMIT 1' clause. The sixty-third section shows a query for 'Teachers' with a 'LIMIT 1' clause. The sixty-fourth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The sixty-fifth section shows a query for 'Teachers' with a 'LIMIT 1' clause. The sixty-sixth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The sixty-seventh section shows a query for 'Teachers' with a 'LIMIT 1' clause. The sixty-eighth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The sixty-ninth section shows a query for 'Teachers' with a 'LIMIT 1' clause. The seventieth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The seventy-first section shows a query for 'Teachers' with a 'LIMIT 1' clause. The seventy-second section shows a query for 'Faculties' with a 'LIMIT 1' clause. The seventy-third section shows a query for 'Teachers' with a 'LIMIT 1' clause. The seventy-fourth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The seventy-fifth section shows a query for 'Teachers' with a 'LIMIT 1' clause. The seventy-sixth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The seventy-seventh section shows a query for 'Teachers' with a 'LIMIT 1' clause. The seventy-eighth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The seventy-ninth section shows a query for 'Teachers' with a 'LIMIT 1' clause. The eightieth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The eighty-first section shows a query for 'Teachers' with a 'LIMIT 1' clause. The eighty-second section shows a query for 'Faculties' with a 'LIMIT 1' clause. The eighty-third section shows a query for 'Teachers' with a 'LIMIT 1' clause. The eighty-fourth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The eighty-fifth section shows a query for 'Teachers' with a 'LIMIT 1' clause. The eighty-sixth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The eighty-seventh section shows a query for 'Teachers' with a 'LIMIT 1' clause. The eighty-eighth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The eighty-ninth section shows a query for 'Teachers' with a 'LIMIT 1' clause. The ninetieth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The ninety-first section shows a query for 'Teachers' with a 'LIMIT 1' clause. The ninety-second section shows a query for 'Faculties' with a 'LIMIT 1' clause. The ninety-third section shows a query for 'Teachers' with a 'LIMIT 1' clause. The ninety-fourth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The ninety-fifth section shows a query for 'Teachers' with a 'LIMIT 1' clause. The ninety-sixth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The ninety-seventh section shows a query for 'Teachers' with a 'LIMIT 1' clause. The ninety-eighth section shows a query for 'Faculties' with a 'LIMIT 1' clause. The ninety-ninth section shows a query for 'Teachers' with a 'LIMIT 1' clause. The hundredth section shows a query for 'Faculties' with a 'LIMIT 1' clause.



Security Testing

Table 9.11 Examples of security risks

Unauthorized attacker gains access to a system using authorized credentials.

Authorized individual accesses resources that are forbidden to that person.

Authentication system fails to detect unauthorized attacker.

Attacker gains access to database using SQL poisoning attack.

Improper management of HTTP sessions.

HTTP session cookies are revealed to an attacker.

Confidential data are unencrypted.

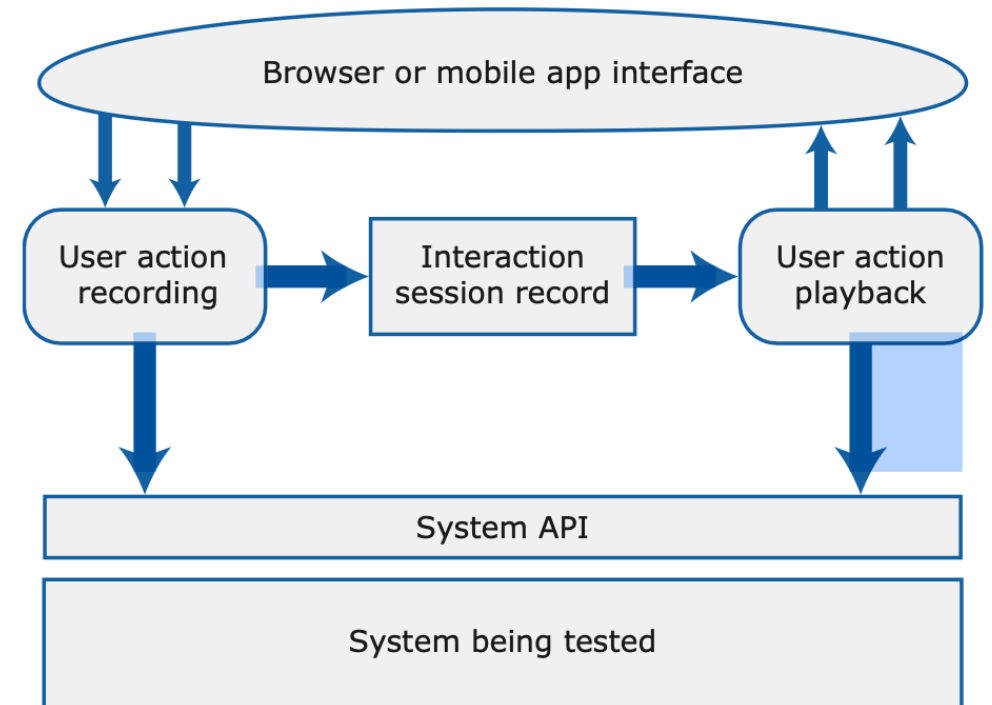
Encryption keys are leaked to potential attackers.



Testing Techniques – Record and Playback

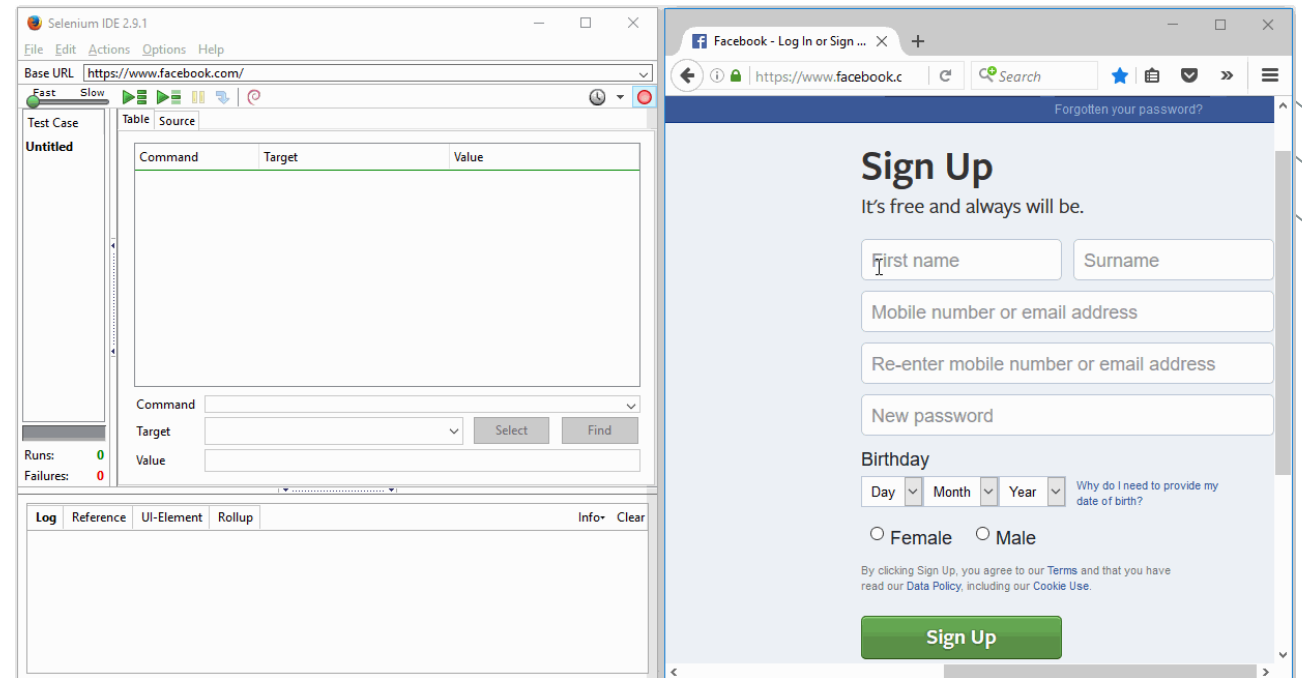
Figure 9.7 Interaction recording and playback

- Suitable for End-To-End Testing
- It tests an entire functionality across the system
- End of the Cycle Technique
- Example: New Account Registration process from front-end.
- Tool:



Testing Techniques – Record and Playback

- Suitable for End-To-End Testing
- It tests an entire functionality across the system
- End of the Cycle Technique
- Example: New Account Registration process from front-end.
- Tool:





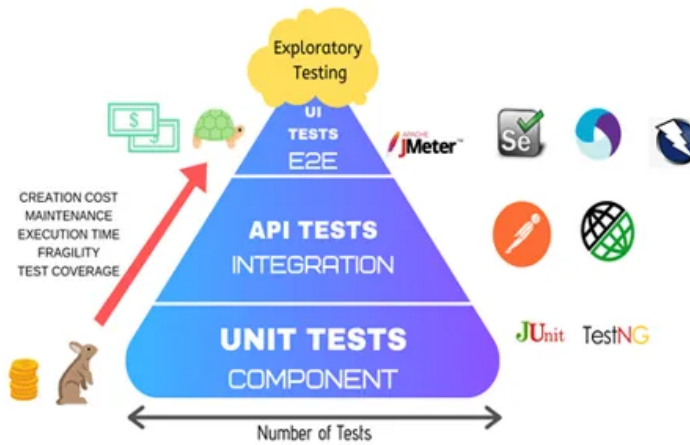
Matching testing types with levels

	Functional Testing	User Testing	Performance Testing	Security Testing
Unit Testing	Partition testing / Guideline based testing	Storybook / Visual Testing (A/B Testing)	Stress testing	Partition testing
Integration Testing	System Testing / Behaviour Driven Testing	Requirements-based testing (Acceptance testing)	Interface Testing	?
End-to-End testing	Release Testing / Behaviour Driven Testing	Manual Scenario Testing	Record and Replay	Third-party White-hat hacking



Designing a Testing Approach for your Software

Test Pyramid Approach *Aim is to Prevent Bugs*

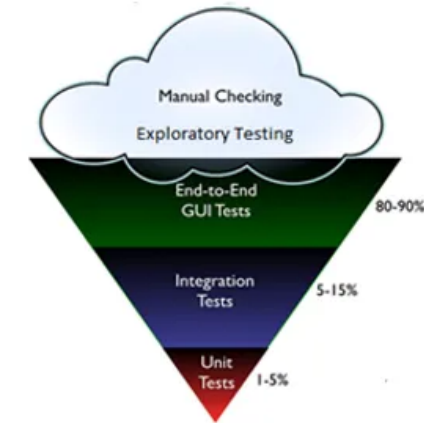


Tool to fix the problem of over-reliance on long-running UI tests.

Detects bugs earlier which reduce project cost and improve overall quality of the Application.

Test Automation (Unit Test, API Test & UI Test)
Time is low.

Traditional (Ice-cream Cone) Approach *Aim is to Find Bugs*



Number of end-to-end tests required to get a good coverage is much higher and more complex than the number of unit tests plus selected end-to-end tests required.

Invest in automation, but mostly building slow, complex, fragile end-to-end GUI test.

Software Engineer's Role in Testing



Software Engineers

Design Documentation → Design
Data Structures → Design
Architecture →

Write Functional Code →
Review Code →
Unit Test + Integration Test →



Software Engineers in Test

Review the Design →
Check Code Quality & Risks →
Refactor the Code to make it more
testable →

Write Unit Testing Frameworks &
Automation →
Increase Quality & Test Coverage by
adding new features →
Work with Developers →



Test Engineers

Test with their deep Domain
knowledge & Product expertise →
Create / Execute Automation
Scripts →

Report Bugs →
Act as End User & Analyze Risks →

How to understand Software Testing better?



Software Testing Guide

I grew up in the waterfall era, where testing was seen as a separate activity to programming, done by a different group of people, and carried out after programming was done. The shift towards iterative and agile approaches, particularly the influence of Extreme Programming, has changed the role of testing - raising its importance, and integrating it with the core development work.

At the core of my philosophy to testing is that we want to write self-testing code, meaning we have a suite of automated tests that be easily run against the code base. We should be confident that this suite will catch almost all bugs in the software, so that when this suite “goes green” (passes), we can release that version of the software into production. This kind of environment can both drastically improve the robustness of our software, but also enables us to use refactoring to improve our productivity and responsiveness in the coming months.

A guide to material on martinofowler.com about testing software systems.

Martin Fowler

2 Dec 2019

Self Testing Code

Specification By Example

Test Driven Development



Kahoot!



Software Testing Requirements

- Implement a Software Testing Process for your project.
- Use the most appropriate testing process
 - Explain why it was chosen (compare with other methodologies)
 - Describe its components
- Define tests on all three levels (end-to-end, integration, unit)
 - Alternatively, argue why on your project one level is not appropriate.
- The scenarios that you are going to present at the live demo, should be thoroughly tested.
 - Identify the most critical qualities of your software
 - Define test suites that are appropriate for those qualities

Your project must prove that you:

- Can differentiate between multiple testing types
- Know when to apply each
- Implement them, so that they are useful for the application.



Resources Used

